

Arquitectura de Persistencia Políglota para un Sistema de Gestión de Inventario Rural

Semanas 14–16 — Integración, Consolidación y Cierre

Santiago Chavarro Herrera
Grupo 10
schavarro@unbosque.edu.co

Juan Pablo Sánchez
Grupo 10
jpasanchez@unbosque.edu.co

Resumen—Este artículo presenta el diseño, implementación y validación de un prototipo de sistema de gestión de inventario orientado a distribuidores rurales de productos agrícolas del departamento del Huila (café, panela, cacao, miel), integrando dos paradigmas de bases de datos: relacional (PostgreSQL) y documental (MongoDB), constituyendo una arquitectura de persistencia políglota. La propuesta aprovecha las fortalezas complementarias de cada tecnología: las garantías ACID del modelo relacional para datos transaccionales críticos, y la flexibilidad BASE/CAP del modelo documental para información dinámica y semiestructurada. Se documentan las decisiones de diseño mediante una Matriz de Decisiones de Entidades, el esquema de datos, los mecanismos de automatización mediante *triggers*, la implementación de un demo técnico interactivo como evidencia de funcionamiento, y una reflexión crítica sobre el proceso de aprendizaje. El trabajo demuestra que la selección deliberada del motor de persistencia según la naturaleza de cada dato produce sistemas más robustos, escalables y mantenibles que una solución monolítica.

Index Terms—PostgreSQL, MongoDB, persistencia políglota, ACID, BASE, CAP, triggers, inventario rural, arquitectura híbrida, demo técnico.

I. INTRODUCCIÓN Y PROPUESTA DEL PROYECTO

En el contexto actual de digitalización de procesos empresariales, las organizaciones requieren sistemas capaces de manejar datos heterogéneos: algunos altamente estructurados y con exigencias transaccionales estrictas, y otros variables, semiestructurados o de alta velocidad de generación. Esta heterogeneidad plantea un desafío de diseño que una única tecnología de base de datos rara vez resuelve de forma óptima [1].

I-A. Planteamiento del problema

Las soluciones exclusivamente relacionales imponen esquemas rígidos que dificultan el registro de información variable — atributos específicos de lotes de producción, metadatos de eventos, configuraciones dinámicas — y pueden convertirse en cuellos de botella ante cargas de escritura intensiva. Por otro lado, las soluciones puramente documentales sacrifican la integridad referencial y las garantías transaccionales indispensables para el control de inventario y la facturación.

Este problema se hace particularmente evidente en el contexto de distribuidores rurales del Huila: un lote de café pergamino de la Vereda El Paraíso requiere registrar variedad,

altitud y proceso de beneficio; un lote de cacao de la Asociación Cacaotera del Macizo necesita días de fermentación y código de trazabilidad; un frasco de miel del Apiario El Cedro necesita valor Brix y tipo de flora. Esta variabilidad estructural hace inviable un esquema relacional rígido para los datos de producción.

I-B. Propuesta: Persistencia Políglota

Este proyecto propone el diseño e implementación de un **sistema de gestión de inventario rural** que demuestre una arquitectura híbrida fundamentada en el principio de *persistencia políglota* [1]: asignar a cada tipo de dato el motor más adecuado para su naturaleza, sus patrones de acceso y sus requisitos de consistencia.

I-C. Pilares tecnológicos

- **PostgreSQL** (motor relacional): gestiona la información estructurada y transaccional — usuarios, productos, clientes, ventas e inventario — bajo el modelo ACID. Se aprovechan las restricciones de integridad referencial y los *triggers* para automatizar lógica de negocio crítica directamente en la base de datos [4].
- **MongoDB** (motor documental): almacena información flexible y de esquema variable — lotes de producción con atributos dinámicos e historial de eventos del sistema — bajo el modelo BASE, tolerando consistencia eventual a cambio de mayor disponibilidad y escalabilidad horizontal [3].

I-D. Alcance del prototipo

- Registro y administración de usuarios, productos, clientes y ventas.
- Control automático de inventario mediante *triggers* en PostgreSQL.
- Registro flexible de lotes de producción con atributos dinámicos en MongoDB.
- Historial trazable de eventos del sistema para auditoría.
- Demo técnico interactivo que valida el funcionamiento en vivo de ambas capas.
- Reportes básicos: estado del inventario, ventas por período, productos más vendidos.

La integración entre ambos motores se realiza mediante referencias de identificadores — sin duplicar información — y la lógica de coordinación reside en la capa de aplicación.

II. FUNDAMENTOS TEÓRICOS: ACID, BASE Y EL TEOREMA CAP

Antes de justificar las decisiones de diseño, es necesario establecer los conceptos que las fundamentan.

II-A. Modelo ACID (PostgreSQL)

El acrónimo ACID describe las propiedades que garantizan la fiabilidad de las transacciones en bases de datos relacionales [4]:

- **Atomicidad:** la transacción se ejecuta en su totalidad o no se ejecuta en absoluto.
- **Consistencia:** toda transacción lleva la base de datos de un estado válido a otro estado válido.
- **Aislamiento:** las transacciones concurrentes se ejecutan como si fueran secuenciales.
- **Durabilidad:** una transacción confirmada persiste incluso ante fallos del sistema.

Estas propiedades son esenciales para datos transaccionales como ventas e inventario, donde una operación a medias produciría inconsistencias graves (p.ej., descuento de stock sin registro de venta).

II-B. Modelo BASE y Teorema CAP (MongoDB)

El modelo BASE (*Basically Available, Soft-state, Eventually consistent*) describe el comportamiento de muchos sistemas NoSQL [1]. A diferencia de ACID, acepta que la información puede no ser consistente en todo momento, priorizando la disponibilidad y la tolerancia a particiones de red.

El **Teorema CAP** (Brewer, 2000) establece que un sistema distribuido solo puede garantizar simultáneamente dos de tres propiedades: **Consistencia** (*Consistency*), **Disponibilidad** (*Availability*) y **Tolerancia a particiones** (*Partition tolerance*). MongoDB elige AP (disponibilidad + tolerancia a particiones), siendo adecuado para datos cuya consistencia eventual es aceptable, como el historial de eventos o los atributos de lotes.

II-C. Justificación de la arquitectura híbrida

La combinación de ambos modelos permite obtener lo mejor de cada paradigma: integridad transaccional donde es crítica (ACID en PostgreSQL) y flexibilidad y escalabilidad donde la consistencia eventual es suficiente (BASE/CAP en MongoDB). Esta es la esencia de la persistencia polígota.

III. MATRIZ DE DECISIONES DE ENTIDADES

La Tabla I presenta la clasificación de cada entidad del sistema, el motor asignado y la justificación técnica basada en los principios ACID, BASE y el Teorema CAP.

III-A. Resumen de la clasificación

De las ocho entidades del sistema, **seis residen en PostgreSQL** por sus requisitos de consistencia fuerte e integridad referencial, y **dos residen en MongoDB** por su naturaleza semiestructurada y los requisitos de flexibilidad de esquema. La integración se realiza mediante referencias de `producto_id` y `usuario_id` en los documentos de MongoDB apuntando a registros de PostgreSQL.

III-B. Trade-offs de la arquitectura

La elección de una arquitectura híbrida introduce compromisos que deben gestionarse conscientemente:

- **Complejidad operativa:** se deben administrar y monitorear dos motores distintos, con diferentes modelos de backup, autenticación y configuración.
- **Consistencia entre motores:** no existe una transacción distribuida que abarque PostgreSQL y MongoDB simultáneamente. La lógica de compensación (p.ej., registrar un evento en MongoDB solo si la venta en PostgreSQL fue confirmada) recae en la capa de aplicación.
- **Latencia adicional:** operaciones que requieren datos de ambos motores implican dos consultas de red en lugar de una.
- **Beneficio neto:** estos costos son asumibles dado el aumento en flexibilidad, escalabilidad y rendimiento que aporta cada motor en su dominio.

IV. DESCRIPCIÓN GENERAL DEL SISTEMA

El sistema se compone de dos capas de persistencia integradas bajo una única capa de aplicación:

- **Capa relacional (PostgreSQL):** gestiona la información estructurada y transaccional — usuarios, productos, clientes, ventas e inventario — garantizando integridad referencial mediante claves primarias, foráneas y restricciones.
- **Capa documental (MongoDB):** almacena datos flexibles — lotes de producción con atributos variables e historial de eventos — bajo consistencia eventual.

La integración entre ambas capas se realiza mediante referencias de identificadores: los documentos en MongoDB incluyen campos `producto_id` o `usuario_id` que apuntan a registros en PostgreSQL, evitando duplicación de información y manteniendo un único punto de verdad para cada entidad.

V. BASE DE DATOS RELACIONAL — POSTGRESQL

V-A. Esquema de tablas

El modelo relacional está compuesto por seis tablas cuyas responsabilidades se describen en la Tabla II. Los datos de prueba utilizan productos reales del sistema de distribución rural del Huila: Café Pergamino Húmedo, Café Tostado Molido, Panela en Bloque, Miel de Abejas y Cacao en Grano Seco; y clientes como la Cooperativa Huila Verde (Pitalito), Distribuidora El Macizo (San Agustín) y Tienda Rural La Cosecha (Acevedo).

Cuadro I
MATRIZ DE DECISIONES DE PERSISTENCIA POLÍGLOTA

Entidad	Motor	Modelo	CAP	Justificación técnica
usuario	PostgreSQL	ACID	CP	Requiere unicidad de email (UNIQUE) y autenticación confiable. La integridad es crítica: un usuario duplicado o inconsistente compromete el control de acceso. Cambios deben confirmarse antes de ser visibles (consistencia fuerte).
cliente	PostgreSQL	ACID	CP	Entidad transaccional que se referencia en cada venta (FK). La integridad referencial garantiza que no existan ventas huérfanas. El volumen es bajo y la estructura es estable, favoreciendo el esquema relacional.
producto	PostgreSQL	ACID	CP	Núcleo del sistema de inventario. Su precio y disponibilidad deben ser consistentes en todas las operaciones de venta simultáneas. El constraint CHECK (precio >= 0) previene datos inválidos.
inventario	PostgreSQL	ACID	CP	Entidad con los requisitos de consistencia más exigentes: el descuento de stock debe ser atómico con el registro del detalle de venta. Un fallo a mitad de la operación causaría stock negativo o ventas sin respaldo. Los triggers garantizan esta atomicidad.
venta	PostgreSQL	ACID	CP	Transacción financiera central. El campo total se recalcula automáticamente vía trigger, garantizando consistencia. El estado (pendiente/completada/cancelada) debe reflejarse de forma inmediata y consistente.
detalle_venta	PostgreSQL	ACID	CP	Tabla de resolución N:M entre ventas y productos. Cada inserción activa el trigger de descuento de stock, haciendo que ambas operaciones sean atómicas. Sin ACID, sería posible vender un producto sin descontarlo del inventario.
lotes_produccion	MongoDB	BASE	AP	Los atributos de un lote varían según el tipo de producto agrícola (café: variedad, altitud, proceso; cacao: días de fermentación, trazabilidad; miel: brix, flora; panela: punto de cocción, variedad de caña). Un esquema relacional rígido requeriría decenas de columnas opcionales. El campo datos_variables del modelo documental permite registrar cualquier combinación sin alterar la estructura ni requerir migraciones. La consistencia eventual es aceptable: un lote registrado segundos después no compromete operaciones transaccionales.
eventos_historial	MongoDB	BASE	AP	Log de auditoría de alta frecuencia de escritura. Los eventos son inmutables una vez registrados y su estructura varía (tipo, metadata, referencias). No requieren consultas relacionales complejas. MongoDB ofrece escalabilidad horizontal nativa para colecciones de alto volumen de inserción, y la consistencia eventual es suficiente para un registro histórico de auditoría.

Leyenda — Motor: **PostgreSQL** = modelo relacional (CP: Consistencia + Tolerancia a particiones); **MongoDB** = modelo documental (AP: Disponibilidad + Tolerancia a particiones).

Cuadro II
TABLAS DEL MODELO RELACIONAL

Tabla	Descripción
usuario	Gestiona el acceso al sistema. Cada usuario administra productos.
cliente	Personas o cooperativas que realizan compras. Se relacionan con las ventas.
producto	Artículos disponibles para venta. Pertenecen a un usuario (FK).
inventario	Control de stock por producto. Relación 1:1 con producto.
venta	Registra transacciones. El total se calcula automáticamente.
detalle_venta	Tabla de unión que resuelve la relación N:M entre ventas y productos.

V-B. Relaciones principales

- Un *usuario* gestiona uno o varios *productos* (1:N).
- Un *producto* tiene exactamente un registro de *inventario* (1:1).

- Un *cliente* puede realizar múltiples *ventas* (1:N).
- Una *venta* contiene uno o varios *productos* a través de *detalle_venta* (N:M).

V-C. Definición del esquema

Listing 1. Tablas principales del modelo relacional

```

1 CREATE TABLE usuario (
2   id SERIAL PRIMARY KEY,
3   nombre VARCHAR(100) NOT NULL,
4   email VARCHAR(150) NOT NULL UNIQUE,
5   password VARCHAR(255) NOT NULL,
6   created_at TIMESTAMP DEFAULT NOW()
7 );
8
9 CREATE TABLE producto (
10  id SERIAL PRIMARY KEY,
11  usuario_id INT NOT NULL
12  REFERENCES usuario(id) ON DELETE RESTRICT,
13  nombre VARCHAR(150) NOT NULL,
14  descripcion TEXT,
15  precio NUMERIC(10,2) NOT NULL CHECK (precio >= 0)
16 );
17
18 CREATE TABLE inventario (
19  id SERIAL PRIMARY KEY,
20  producto_id INT NOT NULL UNIQUE

```

```

21 REFERENCES producto(id) ON DELETE
22 CASCADE,
23 cantidad INT NOT NULL DEFAULT 0,
24 cantidad_minima INT NOT NULL DEFAULT 5,
25 updated_at TIMESTAMP DEFAULT NOW()
26 );
27 CREATE TABLE venta (
28 id SERIAL PRIMARY KEY,
29 cliente_id INT NOT NULL
30 REFERENCES cliente(id) ON DELETE RESTRICT,
31 fecha TIMESTAMP DEFAULT NOW(),
32 total NUMERIC(12,2) DEFAULT 0,
33 estado VARCHAR(20) DEFAULT 'pendiente'
34 CHECK (estado IN
35 ('pendiente', 'completada', 'cancelada'))
36 );
37 CREATE TABLE detalle_venta (
38 id SERIAL PRIMARY KEY,
39 venta_id INT NOT NULL
40 REFERENCES venta(id) ON DELETE CASCADE,
41 producto_id INT NOT NULL
42 REFERENCES producto(id) ON DELETE
43 RESTRICT,
44 cantidad INT NOT NULL CHECK (cantidad
45 > 0),
46 precio_unitario NUMERIC(10,2) NOT NULL

```

```

6 SELECT cantidad INTO stock_actual
7 FROM inventario
8 WHERE producto_id = NEW.producto_id;
9
10 IF stock_actual < NEW.cantidad THEN
11 RAISE EXCEPTION
12 'Stock_insuficiente._Producto_id=%,
13 _stock_actual=%,_pedido=%',
14 NEW.producto_id, stock_actual, NEW.cantidad;
15 END IF;
16
17 UPDATE inventario
18 SET cantidad = cantidad - NEW.cantidad
19 WHERE producto_id = NEW.producto_id;
20
21 RETURN NEW;
22 END;
23 $$ LANGUAGE plpgsql;

```

VI-C. *trg_recalcular_total* — Consistencia del total de venta

Evento: AFTER INSERT OR UPDATE OR DELETE sobre detalle_venta.

Propósito: recalcula el campo total de la venta sumando cantidad × precio_unitario de todos sus ítems. Garantiza que el total nunca quede desincronizado con el contenido real de la venta.

Listing 4. Trigger de recálculo de total de venta

```

1 CREATE OR REPLACE FUNCTION fn_recalcular_total()
2 RETURNS TRIGGER AS $$
3 DECLARE v_id INT;
4 BEGIN
5     v_id := COALESCE(NEW.venta_id, OLD.venta_id);
6     UPDATE venta
7     SET total = (
8         SELECT COALESCE(SUM(cantidad * precio_unitario), 0)
9         FROM detalle_venta WHERE venta_id = v_id
10     )
11     WHERE id = v_id;
12     RETURN COALESCE(NEW, OLD);
13 END;
14 $$ LANGUAGE plpgsql;

```

VI. TRIGGERS IMPLEMENTADOS EN POSTGRESQL

Los *triggers* son funciones que PostgreSQL ejecuta automáticamente ante eventos de modificación en las tablas. Se implementaron cuatro triggers que encapsulan la lógica de negocio directamente en la base de datos, garantizando su ejecución con independencia del cliente o la capa de aplicación que acceda al sistema [2].

VI-A. *trg_inventario_updated* — Timestamp de modificación

Evento: BEFORE UPDATE sobre inventario.

Propósito: actualiza automáticamente el campo `updated_at` con la fecha y hora exacta de cada modificación al stock.

Listing 2. Trigger de actualización de timestamp

```

1 CREATE OR REPLACE FUNCTION fn_inventario_updated()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     NEW.updated_at = NOW();
5     RETURN NEW;
6 END;
7 $$ LANGUAGE plpgsql;

```

VI-B. *trg_descontar_stock* — Validación y descuento de inventario

Evento: AFTER INSERT sobre detalle_venta.

Propósito: al registrar un producto en una venta, descuenta automáticamente la cantidad vendida del inventario. Si el stock disponible es insuficiente, cancela la transacción completa mediante excepción, protegiendo la integridad del inventario. Esta es la implementación más crítica del principio ACID en el sistema.

Listing 3. Trigger de descuento de stock

```

1 CREATE OR REPLACE FUNCTION fn_descontar_stock()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     stock_actual INT;
5 BEGIN

```

VI-D. *trg_alerta_stock* — Monitoreo de stock mínimo

Evento: AFTER UPDATE sobre inventario.

Propósito: tras cada actualización de stock, verifica si la cantidad actual cayó por debajo del mínimo configurado y emite un aviso (NOTICE) que la capa de aplicación puede capturar para disparar notificaciones.

Listing 5. Trigger de alerta de stock mínimo

```

1 CREATE OR REPLACE FUNCTION fn_alerta_stock()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF NEW.cantidad <= NEW.cantidad_minima THEN
5         RAISE NOTICE
6         'ALERTA_STOCK_BAJO._Producto_id=%,
7         _stock_actual=%,_minimo=%',
8         NEW.producto_id,
9         NEW.cantidad,
10        NEW.cantidad_minima;
11     END IF;
12     RETURN NEW;
13 END;
14 $$ LANGUAGE plpgsql;

```

La Tabla III resume los cuatro triggers implementados y su relación con los principios ACID.

Cuadro III
RESUMEN DE TRIGGERS IMPLEMENTADOS

Trigger	Evento	Propósito / Principio ACID
trg_inventario_updated	BEFORE UPDATE	Timestamp automático — Durabilidad
trg_descontar_stock	AFTER INSERT	Valida stock y descuento — Atomicidad
trg_recalcular_total	AFTER I/U/D	Total consistente — Consistencia
trg_alerta_stock	AFTER UPDATE	Monitoreo proactivo — Consistencia

VII. BASE DE DATOS DOCUMENTAL — MONGODB

MongoDB gestiona los datos que, por su naturaleza variable o su alto volumen de escritura, no se adaptan eficientemente a un esquema relacional rígido [3]. La decisión de usar MongoDB para estas entidades responde directamente al principio CAP: se acepta consistencia eventual (AP) a cambio de disponibilidad y flexibilidad de esquema.

VII-A. Colección: *lotes_produccion*

Almacena cada lote fabricado o recibido de un producto. El campo `datos_variables` es un objeto libre que permite registrar atributos específicos según el tipo de producto sin alterar la estructura general del documento ni requerir migraciones de esquema. La Tabla IV ilustra la heterogeneidad de atributos por tipo de producto rural.

Cuadro IV
EJEMPLOS DE DATOS_VARIABLES POR TIPO DE PRODUCTO

Producto	Atributos en <code>datos_variables</code>
Café pergamino	variedad, altitud_msnm, temperatura_almacen, proceso
Café tostado	nivel_tostion, granulometria, empaque, fecha_vencimiento
Cacao en grano	dias_fermentacion, humedad_porcentaje, codigo_trazabilidad
Panela en bloque	variedad_cana, rendimiento_litros_kg, punto_coccion
Miel de abejas	tipo_flora, humedad_porcentaje, brix, lote_apicultor

Campos principales: `producto_id` (ref. PostgreSQL), `usuario_id` (ref. PostgreSQL), `fecha`, `cantidad_producida`, `calidad`, `origen`, `notas`, `datos_variables`.

Justificación CAP: la disponibilidad de esta colección es prioritaria (lectura frecuente en reportes de producción). La consistencia eventual es aceptable: si un lote tarda milisegundos en replicarse entre nodos, no afecta las transacciones de venta que ocurren en PostgreSQL.

VII-B. Colección: *eventos_historial*

Funciona como un log inmutable de auditoría del sistema. Registra qué ocurrió, cuándo, qué usuario lo hizo y a qué

registro de PostgreSQL hace referencia. Su volumen crece continuamente y su estructura varía por tipo de evento.

Tipos de evento: `produccion`, `venta`, `ajuste_inventario`, `devolucion`, `alerta_stock`.

Campos principales: `tipo`, `fecha`, `descripcion`, `referencia_id`, `producto_id`, `usuario_id`, `metadata`.

Justificación BASE: los eventos son escrituras de alta frecuencia donde la disponibilidad de inserción es prioritaria. Una vez confirmada la transacción en PostgreSQL, el evento se registra en MongoDB de forma asíncrona; si falla, puede reintentarse sin comprometer la integridad del inventario.

VII-C. Índices implementados

Para optimizar las consultas más frecuentes sobre ambas colecciones se definieron los siguientes índices:

Listing 6. Índices de MongoDB para *lotes_produccion* y *eventos_historial*

```

1 // lotes_produccion
2 db.lotes_produccion.createIndex({ producto_id: 1 });
3 db.lotes_produccion.createIndex({ fecha: -1 });
4 db.lotes_produccion.createIndex({ producto_id: 1, fecha: -1
5 });
6 db.lotes_produccion.createIndex({ usuario_id: 1 });
7
8 // eventos_historial
9 db.eventos_historial.createIndex({ tipo: 1 });
10 db.eventos_historial.createIndex({ fecha: -1 });
11 db.eventos_historial.createIndex({ producto_id: 1, fecha:
12 -1 });
13 db.eventos_historial.createIndex({ referencia_id: 1 });
14 db.eventos_historial.createIndex({ usuario_id: 1 });

```

VIII. REPORTES

El sistema expone reportes a través de vistas en PostgreSQL y consultas de agregación en MongoDB, minimizando la lógica en el backend:

- **v_inventario:** estado del stock con alerta automática cuando cae por debajo del mínimo.
- **v_ventas_detalle:** ventas con nombre de cliente, productos y totales.
- **v_top_productos:** productos más vendidos en unidades e ingresos (solo ventas completadas).
- **Historial de producción:** consulta sobre `lotes_produccion` filtrada por `producto_id`.
- **Log de eventos:** historial completo desde `eventos_historial`, filtrable por tipo o rango de fechas.
- **Agregación de producción:** total producido por producto con número de lotes, implementada como pipeline de agregación en MongoDB.

IX. DEMO TÉCNICO INTERACTIVO

Como evidencia de funcionamiento del sistema, se implementó un demo técnico en Python que conecta simultáneamente a PostgreSQL y MongoDB, permitiendo demostrar en vivo las operaciones CRUD completas y el comportamiento de los triggers ACID.

IX-A. Arquitectura del demo

El demo fue desarrollado con las librerías `psycopg2` (conector PostgreSQL), `pymongo` (conector MongoDB) y `rich` (interfaz de terminal visual). Se estructura en un menú interactivo con las siguientes secciones:

Cuadro V
MÓDULOS DEL DEMO TÉCNICO INTERACTIVO

Módulo	Operaciones disponibles
PG — Consultas	Productos, clientes, ventas, detalle de venta, alertas de stock
PG — Insertar	Nuevo producto (con stock), nuevo cliente, nueva venta interactiva
PG — Actualizar	Ajustar stock, cambiar estado de venta, cambiar precio
PG — Eliminar	Eliminar cliente, venta o producto (respetando FK)
Trigger ACID	Demo en vivo de atomicidad: intento de venta con stock insuficiente
Mongo — Consultas	Lotes por producto, historial de eventos, alertas, agregación
Mongo — Insertar	Nuevo lote con datos_variáveis, nuevo evento manual
Mongo — Eliminar	Por producto, por tipo de evento, último documento
Arquitectura	Resumen visual de trade-offs ACID vs BASE/-CAP
Dashboard HTML	Exportar resultados a panel web interactivo

IX-B. Demo del trigger ACID — Atomicidad en vivo

La demostración más crítica del sistema prueba el `trg_descontar_stock` intentando insertar una cantidad mayor al stock disponible. El siguiente fragmento muestra el flujo implementado en el demo:

Listing 7. Fragmento del demo de trigger ACID en Python

```
1 def demo_trigger():
2     c = cur()
3     # consultar stock antes
4     c.execute("SELECT cantidad FROM inventario
5     .....WHERE producto_id=1")
6     stock_antes = c.fetchone()[ "cantidad" ]
7     # intentar insertar 999999 unidades
8     try:
9         c.execute("""INSERT INTO detalle_venta
10        .....(venta_id, producto_id,
11        .....cantidad, precio_unitario)
12        .....VALUES (1, 1, 999999, 12500) """)
13         PG_CONN.commit()
14     except Exception as e:
15         PG_CONN.rollback() # ROLLBACK automatico
16         # verificar que el stock no cambio
17         c2 = cur()
18         c2.execute("SELECT cantidad FROM inventario
19        .....WHERE producto_id=1")
20         stock_despues = c2.fetchone()[ "cantidad" ]
21         # stock_antes == stock_despues: ACID verificado
```

Resultado observado: PostgreSQL lanza la excepción `Stock insuficiente`. Producto `id=1`, stock actual=480, pedido=999999 y revierte la transacción completa. El stock permanece en 480 kg, verificando la propiedad de Atomicidad ACID.

IX-C. Integración entre motores en el demo

El demo implementa la coordinación entre capas: cuando se registra una nueva venta (PostgreSQL), automáticamente se inserta un evento de tipo venta en `eventos_historial` (MongoDB). Igualmente, al ajustar stock manualmente en PostgreSQL, el demo registra un evento de `ajuste_inventario` en MongoDB. Esta coordinación simula el comportamiento esperado de una API REST en producción.

Listing 8. Coordinación PostgreSQL-MongoDB al registrar venta

```
1 # 1. Confirmar venta en PostgreSQL (ACID)
2 PG_CONN.commit()
3
4 # 2. Registrar evento en MongoDB (BASE - asincrono)
5 if MONGO_DB is not None:
6     MONGO_DB.eventos_historial.insert_one({
7         "tipo": "venta",
8         "fecha": datetime.datetime.now(),
9         "descripcion": f"Venta_{vid}_completada",
10        "referencia_id": vid,
11        "usuario_id": 1,
12        "metadata": {
13            "productos_vendidos": items
14        }
15    })
```

X. PROPUESTAS DE MEJORAMIENTO

Una vez validado el prototipo, se identifican las siguientes líneas de mejora para una versión de producción:

X-A. Seguridad y autenticación

Implementar autenticación basada en JWT (*JSON Web Tokens*), hash de contraseñas con `bcrypt` y un sistema de roles (administrador, vendedor, supervisor) con permisos diferenciados en ambos motores.

X-B. Backend y API REST

Desarrollar una API REST con Node.js/Express o Python/FastAPI que coordine ambas capas de persistencia, con validación de entrada, manejo centralizado de errores, paginación y lógica de compensación transaccional entre PostgreSQL y MongoDB.

X-C. Optimización de base de datos

Implementar particionamiento de la tabla `venta` por fecha para reportes históricos de alto volumen, índices compuestos en MongoDB para las consultas más frecuentes sobre `eventos_historial`, y un esquema de *soft delete* (campo `deleted_at`) para preservar el historial de registros eliminados.

X-D. Gestión de consistencia entre motores

Implementar un patrón de *outbox* o cola de mensajes (p.ej., RabbitMQ o Kafka) para garantizar que los eventos en MongoDB se registren exactamente una vez cuando la transacción de PostgreSQL confirma, eliminando el acoplamiento directo entre capas.

X-E. Interfaz y experiencia de usuario

Desarrollar un panel web con gráficas de ventas e inventario en tiempo real, notificaciones automáticas por correo ante alertas de stock y exportación de reportes a PDF y Excel.

X-F. Calidad y despliegue

Escribir pruebas de integración para los flujos críticos (descuento de stock, alerta de mínimos, registro de eventos), contenedorizar la aplicación con Docker Compose e implementar un pipeline de CI/CD para automatizar pruebas y despliegues en servicios administrados en la nube (PostgreSQL en Azure/AWS, MongoDB Atlas).

XI. CONCLUSIONES

La arquitectura de persistencia polígota propuesta demuestra que la selección deliberada del motor de base de datos según la naturaleza de cada dato — sus patrones de acceso, requisitos de consistencia y variabilidad de esquema — produce sistemas más robustos y mantenibles que una solución monolítica.

La Matriz de Decisiones evidencia que las seis entidades transaccionales del sistema (usuario, cliente, producto, inventario, venta, detalle_venta) requieren las garantías ACID de PostgreSQL para preservar la integridad del inventario y la facturación, mientras que las dos entidades dinámicas (lotes_produccion, eventos_historial) se benefician del modelo BASE/CAP de MongoDB para absorber la variabilidad de esquema y el alto volumen de escritura sin comprometer el rendimiento.

Los *triggers* implementados en PostgreSQL encapsulan la lógica de negocio crítica directamente en la base de datos, garantizando consistencia independientemente del cliente. El demo técnico interactivo validó en vivo el comportamiento del `trg_descontar_stock`, confirmando la propiedad de Atomicidad ACID ante intentos de venta con stock insuficiente. MongoDB complementa el sistema absorbiendo la variabilidad de los datos de producción e historial — particularmente relevante para los productos agrícolas del Huila, donde cada tipo de producto requiere atributos distintos — sin imponer un esquema rígido.

El prototipo sienta las bases para una solución de producción escalable, cuyas mejoras de mayor impacto son la implementación de un patrón de consistencia entre motores y la adición de un sistema de autenticación robusto.

XII. REFLEXIÓN FINAL DE APRENDIZAJE

XII-A. Del esquema único a la arquitectura deliberada

Al comenzar el semestre, nuestra visión de una base de datos era casi instintiva: si el sistema necesitaba guardar datos, la respuesta era un motor relacional. Era lo que conocíamos, lo que nos enseñaron primero, y lo que nos parecía seguro. El proyecto integrador nos obligó a romper esa certeza.

Diseñar un sistema para distribuidores rurales del Huila nos puso frente a una realidad concreta: los datos del mundo real no son uniformes. Un lote de café pergamino tiene variedad, altitud y proceso de beneficio; un lote de cacao tiene días

de fermentación y código de trazabilidad; un frasco de miel tiene valor Brix y tipo de flora. Intentar guardar todo eso en un esquema relacional rígido significaba elegir entre dos malos caminos: decenas de columnas opcionales llenas de nulos, o tablas auxiliares que complicaban las consultas sin agregar valor. Fue ahí cuando el campo `datos_variables` de MongoDB dejó de ser un concepto de clase y se convirtió en una solución concreta.

XII-B. El Teorema CAP en la práctica

Antes del curso, la idea de que un sistema distribuido no puede garantizar consistencia, disponibilidad y tolerancia a particiones simultáneamente nos habría parecido una limitación académica. Hoy entendemos por qué importa.

Cuando diseñamos los *triggers* de PostgreSQL para el descuento de stock, estábamos aplicando el modelo CP conscientemente: preferimos consistencia fuerte aunque eso signifique lanzar una excepción y revertir toda la transacción. Un distribuidor de café no puede permitirse vender 999.999 kg que no existen. La integridad es innegociable.

Cuando diseñamos `eventos_historial` en MongoDB, aplicamos el modelo AP: si un evento de auditoría tarda milisegundos en replicarse, no pasa nada. La venta ya fue confirmada en PostgreSQL. El historial puede llegar después. Esa distinción, antes abstracta, ahora nos parece natural.

XII-C. Los triggers como pensamiento declarativo

Uno de los cambios más importantes fue entender los *triggers* no como una característica avanzada, sino como una forma distinta de pensar la lógica de negocio. En lugar de decir “el código debe recordar descontar el stock”, el *trigger* dice: “el descuento es parte de la inserción, siempre, sin excepción, independientemente de qué cliente acceda al sistema”. Eso es atomicidad en práctica, no en teoría.

XII-D. Competencias hacia el futuro

Este proyecto dejó dos preguntas que queremos seguir respondiendo: cómo garantizar la consistencia entre dos motores en producción (patrón Outbox con Kafka) y cómo desplegar esto en la nube (PostgreSQL en Azure, MongoDB Atlas, Docker Compose). Salimos del semestre sabiendo no solo usar dos motores distintos, sino **cuándo usar cada uno y por qué**: la capacidad de justificar decisiones técnicas con criterios concretos es la diferencia entre ejecutar instrucciones y diseñar sistemas.

REFERENCIAS

- [1] M. Fowler y P. Sadalage, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional, 2012.
- [2] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2023. [En línea]. Disponible en: <https://www.postgresql.org/docs/16/>
- [3] K. Chodorow, *MongoDB: The Definitive Guide*, 2.ª ed. O'Reilly Media, 2013.
- [4] C. J. Date, *An Introduction to Database Systems*, 8.ª ed. Addison-Wesley, 2004.
- [5] E. A. Brewer, “Towards Robust Distributed Systems,” in *Proc. 19th Annual ACM Symp. on Principles of Distributed Computing (PODC)*, Portland, OR, 2000, p. 7.